Database Schema Document

AI-Powered Construction Site Intelligence Platform

Version 1.1 | PostgreSQL | Status: Engineering-Ready

Design Principles

- All audit tables (violations, delivery_logs) are append-only. No UPDATE or DELETE endpoints are exposed — this is the tamper-evidence requirement.
- All foreign keys use ON DELETE SET NULL for audit tables — so removing a user or gate never deletes historical evidence records.
- discrepancy_flag is computed from vehicle count mismatch only. material_type/material_quantity fields are informational and do not affect fraud logic.
- material_type is constrained to an ENUM/CHECK list to prevent dirty data.
- Indexes are added on high-query fields: detected_at (violations), discrepancy_flag (delivery_logs), category (hazard_reports).

1. Table: users

Stores all actors in the system. Role determines access control across all modules. Admin creates all user accounts — self-registration is not permitted.

Field	Data Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique user identifier
name	VARCHAR(100)	NOT NULL	Full name of user
email	VARCHAR(150)	NOT NULL, UNIQUE	Login credential and notification address
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash — never store plaintext
role	ENUM('admin','supervisor','worker')	NOT NULL	Determines access level across all modules
site_id	UUID	FK → sites.id, ON DELETE SET NULL	Which site this user belongs to (nullable for multi-site admin)
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Soft-delete: deactivated users cannot log in
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Account creation timestamp
last_login_at	TIMESTAMP	NULLABLE	Last successful login — audit trail

Indexes

- users(email) — unique index for login lookup

Sample Data

```
INSERT INTO users VALUES ('u1', 'Rajesh Kumar', 'rajesh@site.com', '<hash>', 'admin', null, true, NOW(), null);
INSERT INTO users VALUES ('u2', 'Meena Rao', 'meena@site.com', '<hash>', 'supervisor', 's1', true, NOW(), null);
```

```
INSERT INTO users VALUES ('u3', 'Arjun Das', 'arjun@site.com', '<hash>',
'worker', 's1', true, NOW(), null);
```

2. Table: sites

Represents a physical construction site. Scopes zones, gates, equipment, and users to their site. Supports multi-site admin oversight.

Field	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique site identifier
name	VARCHAR(150)	NOT NULL	Site name (e.g., "Hyderabad Site A")
address	TEXT	NULLABLE	Physical address
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Site record creation

3. Table: gates

Represents a physical entry/exit gate at a site, tied to a camera. The gate camera serves both PPE verification (Module 1A) and vehicle counting (Module 4A) — the shared pipeline design.

Field	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique gate identifier
site_id	UUID	FK → sites.id, NOT NULL	Which site this gate belongs to
name	VARCHAR(100)	NOT NULL	e.g. "Main Entry Gate", "South Gate"
location	VARCHAR(150)	NULLABLE	Physical location description
camera_id	VARCHAR(100)	NOT NULL	Camera device identifier for pipeline routing
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Whether gate camera is operational
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	

4. Table: zones

Defines restricted/hazardous areas on the site via polygon coordinates. Zone polygon logic is implemented in OpenCV/Python — the database stores the coordinates only.

Field	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	
site_id	UUID	FK → sites.id, NOT NULL	Which site this zone belongs to
name	VARCHAR(100)	NOT NULL	e.g. "Crane Operation Zone", "High Voltage Area"
zone_type	VARCHAR(50)	NOT NULL	crane / high_voltage / excavation / chemical_storage
polygon_coords	JSONB	NOT NULL	Array of x/y coordinate points defining the restricted area polygon
ppe_rules	JSONB	NULLABLE	Zone-specific PPE requirements (e.g., harness required in addition to helmet+vest)
created_by	UUID	FK → users.id (admin only)	Who defined this zone
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	

Sample Data

```
INSERT INTO zones VALUES ('z1', 's1', 'Crane Zone North', 'crane',
'[{"x":100,"y":200},{"x":300,"y":200}]', null, 'u1', NOW());
```

5. Table: violations

The primary safety audit trail. Every PPE violation and restricted-zone breach is stored here. Append-only — no UPDATE or DELETE endpoints. This table is the legal/compliance record.

Field	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	
site_id	UUID	FK → sites.id, NOT NULL	
worker_id	UUID	FK → users.id, ON DELETE SET NULL, NULLABLE	Null if worker unidentified (no RFID/badge)
gate_id	UUID	FK → gates.id, ON DELETE SET NULL, NULLABLE	For gate-based PPE violations
zone_id	UUID	FK → zones.id, ON DELETE SET NULL, NULLABLE	For restricted-zone violations
violation_type	VARCHAR(50)	NOT NULL	no_helmet / no_vest / no_helmet_no_vest / restricted_zone_entry
screenshot_url	TEXT	NOT NULL	S3-compatible URL to violation snapshot image
confidence_score	DECIMAL(4,3)	NULLABLE	Model detection confidence (0.000–1.000) — for transparency
detected_at	TIMESTAMP	NOT NULL	Exact detection timestamp from AI pipeline
acknowledged_by	UUID	FK → users.id, ON DELETE SET NULL, NULLABLE	Supervisor who acknowledged this violation
acknowledged_at	TIMESTAMP	NULLABLE	When it was acknowledged
status	VARCHAR(20)	DEFAULT 'unacknowledged'	unacknowledged / acknowledged

Indexes

- violations(detected_at) — for date-range dashboard queries
- violations(site_id, detected_at) — composite for site-scoped supervisor views
- violations(status) — for unacknowledged alert counts

6. Table: delivery_logs

The core fraud-detection table. ai_vehicle_count is populated automatically and independently by the gate camera pipeline — it is tamper-evident because it is written before any supervisor interaction. manual_reported_count and material fields are entered by the supervisor. discrepancy_flag is computed from count mismatch only.

Field	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	
site_id	UUID	FK → sites.id, NOT NULL	
gate_id	UUID	FK → gates.id, ON DELETE SET NULL, NOT NULL	Which gate camera logged this delivery

Field	Data Type	Constraints	Description
reported_by	UUID	FK → users.id (supervisor), ON DELETE SET NULL	Supervisor who logged the manual count and material info
ai_vehicle_count	INTEGER	NOT NULL, CHECK >= 0	Auto-populated by gate camera pipeline — independent of supervisor input
manual_reported_count	INTEGER	NULLABLE, CHECK >= 0	Entered by supervisor — compared against ai_vehicle_count
material_type	VARCHAR(50)	CHECK IN ('cement','sand','bricks','steel','pipes','other'), NULLABLE	Manually entered by supervisor — what the vehicles were carrying
material_quantity	DECIMAL(10,2)	NULLABLE, CHECK > 0	Manually entered quantity
quantity_unit	VARCHAR(20)	CHECK IN ('bags','tons','cubic_meters','pieces'), NULLABLE	Unit for the quantity value
vehicle_type	VARCHAR(50)	NULLABLE	Optionally AI-classified vehicle type: mixer / tipper / open_bed / flatbed — soft cross-check only
discrepancy_flag	BOOLEAN	NOT NULL, DEFAULT FALSE	True when repeated count mismatches detected — count-based only, not material-based
discrepancy_count	INTEGER	NOT NULL, DEFAULT 0	Running tally of mismatches at this gate by this supervisor — triggers escalation at threshold
logged_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	When this record was created
ai_logged_at	TIMESTAMP	NULLABLE	When AI count was written — may precede manual entry

Indexes

- delivery_logs(discrepancy_flag) — for admin escalation queue
- delivery_logs(site_id, logged_at) — for date-range material inflow reports
- delivery_logs(gate_id, reported_by, logged_at) — for pattern detection across same supervisor + gate

Business Rule Note

discrepancy_count is incremented by the backend service on each new mismatch at the same gate by the same supervisor. When discrepancy_count reaches the configured threshold (default: 3 consecutive mismatches), discrepancy_flag is set to TRUE and an escalation notification is sent to Admin. A single mismatch does not trigger escalation.

Sample Data

```
INSERT INTO delivery_logs VALUES
('d1','s1','g1','u2',12,9,'cement',500.00,'bags','tipper',FALSE,1,NOW(),NOW()
);
```

7. Table: hazard_reports

Stores worker-submitted hazard reports with AI-assigned categories. Append-only. image_url points to S3-stored upload.

Field	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	
site_id	UUID	FK → sites.id, NOT NULL	
worker_id	UUID	FK → users.id, ON DELETE SET NULL, NOT NULL	Worker who submitted the report
description	TEXT	NOT NULL	Worker-entered description of the hazard
location	VARCHAR(255)	NOT NULL	Text description of where the hazard is (e.g., "North scaffolding, Level 3")
image_url	TEXT	NULLABLE	S3-compatible URL to uploaded hazard photo
category	VARCHAR(50)	NULLABLE	AI-assigned: fire_risk / unsafe_wiring / water_leakage / scaffold_damage / structural_cracks / uncategorized
ai_confidence	DECIMAL(4,3)	NULLABLE	Categorization confidence — low confidence → "uncategorized" queue
status	VARCHAR(20)	DEFAULT 'open'	open / acknowledged / resolved
acknowledged_by	UUID	FK → users.id, ON DELETE SET NULL, NULLABLE	
reported_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	

Indexes

- hazard_reports(category) — for supervisor category-filter view
- hazard_reports(site_id, status) — for open hazard count KPI

8. Table: equipment

Tracks construction machinery on site — location, idle time, and last-seen timestamp from camera detection.

Field	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	
site_id	UUID	FK → sites.id, NOT NULL	
equipment_type	VARCHAR(50)	NOT NULL	excavator / bulldozer / crane / cement_mixer / truck
identifier	VARCHAR(100)	NULLABLE	Vehicle registration or site tag number if known
last_location	VARCHAR(150)	NULLABLE	Human-readable location last seen
last_seen_at	TIMESTAMP	NULLABLE	Timestamp of last detection event
idle_minutes	INTEGER	NOT NULL, DEFAULT 0	Consecutive minutes without movement detected
status	VARCHAR(20)	DEFAULT 'active'	active / idle / offline
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Last update from detection pipeline

9. ER Diagram (Text Representation)

The following describes entity relationships. Use a tool like dbdiagram.io or draw.io to render this as a visual ER diagram for your report.

```
sites ||--o{ users : "has"
sites ||--o{ gates : "has"
sites ||--o{ zones : "has"
sites ||--o{ violations : "logged at"
sites ||--o{ delivery_logs : "recorded at"
sites ||--o{ hazard_reports : "reported at"
sites ||--o{ equipment : "tracked at"
users ||--o{ violations : "flagged (worker)"
users ||--o{ violations : "acknowledged (supervisor)"
users ||--o{ delivery_logs : "filed (supervisor)"
users ||--o{ hazard_reports : "submitted (worker)"
gates ||--o{ violations : "detected at"
gates ||--o{ delivery_logs : "recorded at"
zones ||--o{ violations : "breach location"
```

10. Relationship Explanation

Relationship	Type	Explanation
sites → gates	One-to-many	A site has multiple physical entry gates, each with its own camera
sites → zones	One-to-many	A site can have multiple restricted polygon zones
sites → users	One-to-many	Users are scoped to a site (except admin who can be global)
gates → delivery_logs	One-to-many	Each delivery event is associated with a specific gate camera
users (supervisor) → delivery_logs	One-to-many	The supervisor who filed the manual count is linked — essential for conflict-of-interest exclusion from escalation review
zones → violations	One-to-many	Zone-breach violations reference which zone was entered
gates → violations	One-to-many	Gate-based PPE violations reference which gate camera detected them
users (worker) → hazard_reports	One-to-many	Each hazard report is linked to the submitting worker
delivery_logs.reported_by → users	Many-to-one	FK ensures discrepancy escalations can identify and route away from the filing supervisor